

UAV Propeller Performance — Data Science Capstone

A data-driven framework for predicting thrust coefficient, power coefficient and efficiency

1. Understanding the problem

Small Unmanned Aerial Vehicles (UAVs) and the emerging *urban air mobility* sector live or die on propulsion efficiency. A propeller designer must trade **size, power and weight** against **range and endurance**, and the aerodynamic behaviour of a small, low-Reynolds-number propeller is hard to predict from first principles alone. The brief therefore asks for a **data-driven framework** that forecasts the three performance outputs

Output	Symbol	Meaning
Thrust coefficient	C_T	non-dimensional thrust the propeller produces
Power coefficient	$C_P = P / (\rho n^3 D^5)$	non-dimensional shaft power absorbed
Efficiency	η	useful thrust power / shaft power

from **blade geometry** (diameter, pitch, chord & radius distributions, blade count, solidity) and **operating inputs** (advance ratio J , RPM), using both **standalone and ensemble machine-learning** techniques.

These quantities obey a hard physical identity that the data respects row-by-row:

$$\eta = J \cdot C_T / C_P$$

The work is organised exactly as the problem statement lays it out — **Week 1 (SQL)** and **Week 2 (Data Science, Machine Learning, Tableau)**.

1.1 The datasets

Two families of workbooks were supplied in versioned parts and appended into two tidy tables:

- **Experiment data** — `Experiment_vol1/2/3.csv` → **27,495** operating points across **240** propellers. Columns: propeller/blade name, brand, number of blades, diameter, pitch, advance ratio, RPM, and the three outputs.
- **Geometry data** — `Geom_vol1/2.csv` → **2,238** radial stations for **120** blades. Columns: blade name, brand, diameter, pitch, a-dimensional chord c/R , a-dimensional radius r/R , and the blade angle β .

Data-coverage reality (important): the geometry data was delivered in only **two** volumes, and it describes **120** of the **226** distinct 2-blade blades. **106 propellers therefore have no geometry at all**, so their **solidity cannot be computed**. This is not a nuisance — it is the exact missing-value scenario the Machine-Learning task is built around (see §5).

(This format mirrors the public UIUC / APC small-propeller performance database.)

2. Week 1 — SQL

The three experiment volumes are loaded into an in-memory SQLite database as `experiment1`, `experiment2`, `experiment3`. Full SQL in `src/sql/week1_queries.sql`; runner in `src/05_run_sql.py`; results image `outputs/figures/09_sql_summary.png`

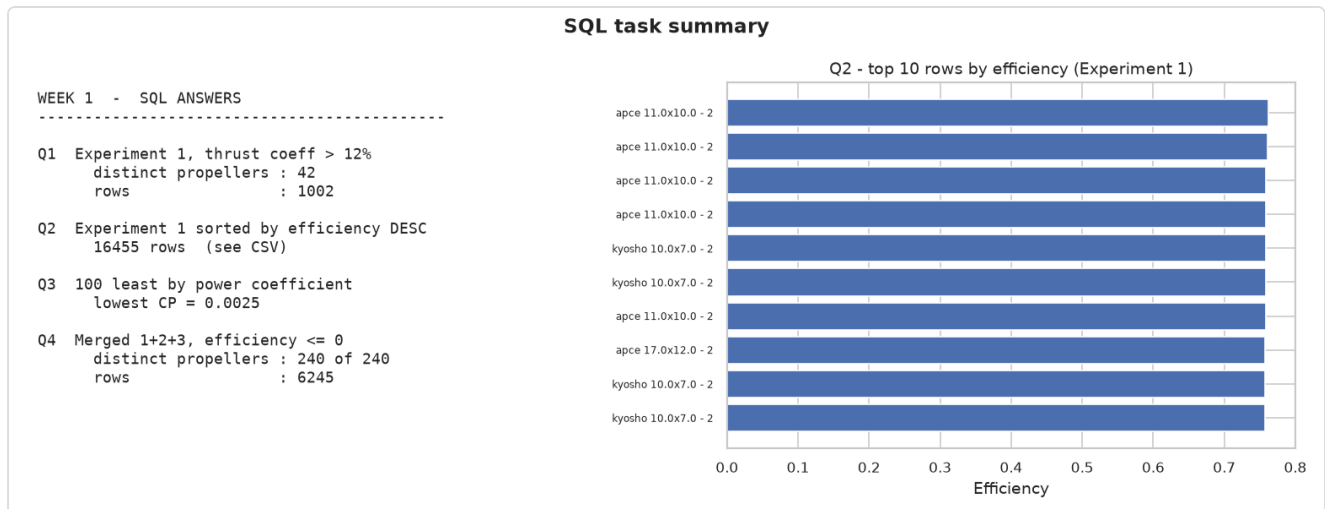


Figure — 09_sql_summary.png

#	Question	Answer
Q1	In experiment 1, number of propellers with thrust coefficient > 12 % ($C_T > 0.12$)	42 propellers (1,002 operating-point rows)
Q2	Reorder experiment 1 by efficiency , descending	16,455 rows sorted; top rows are apce 11.0x10.0 / kyosho 10.0x7.0 at $\eta \approx 0.76$ (sql_q2_efficiency_desc.csv)
Q3	The 100 least-performing propellers by power coefficient in experiment 1	100 lowest- C_P rows saved to sql_q3_least100_power.csv (lowest $C_P \approx 0.0025$)
Q4	Merge experiment 1+2+3; propellers with negative or zero efficiency	240 of 240 propellers (6,245 rows)

Reading Q4: every propeller eventually records $\eta \leq 0$. This is physically correct — beyond its zero-thrust advance ratio a propeller stops producing thrust and begins to windmill/brake, so C_T (and hence η) goes to zero and below. It flags these high-J rows as a data-quality concern for later modelling.

3. Week 2 — Data preparation (src/01_data_preparation.py)

- Append the versions.** Each *_vol*.csv is read and vertically appended, keeping a source_volume provenance column → one experiment table, one geometry table.
- Python-identifier naming.** Every label is converted to a valid PEP-8 identifier by a reusable to_identifier() helper, e.g. "Propeller's Name" → propeller_name, "Adimensional Chord - c/R" → adimensional_chord_c_r, "beta - Angle Relative to Rotation" → beta_angle_relative_to_rotation.
- Physical radius & chord distributions.** With blade radius $R = \text{Diameter}/2$, $\text{radius} = (r/R) \cdot R$ and $\text{chord} = (c/R) \cdot R$ are computed for **every station of every blade** and stored as radius_distribution / chord_distribution.

4. Week 2 — Blade area, disc area & solidity (src/02_solidity_analysis.py)

- Blade area** — the definite integral $A_{\text{blade}} = \int c(r) \, dr$ along the span, evaluated with the **composite trapezoidal rule** via numpy.trapz. Blades that list a radial station more than once are collapsed (mean chord per unique radius) so the integrator sees a clean monotonic span.
- Total blade area** — $\text{number_of_blades} \times A_{\text{blade}}$.
- Disc area** — $A_{\text{disc}} = \pi r^2$, with $r = D/2$.
- Solidity** — $\sigma = \text{total blade area} / \text{disc area}$ (dimensionless).

4.1 Findings — could we compute solidity for every propeller?

See `outputs/figures/01_solidity_findings.png`

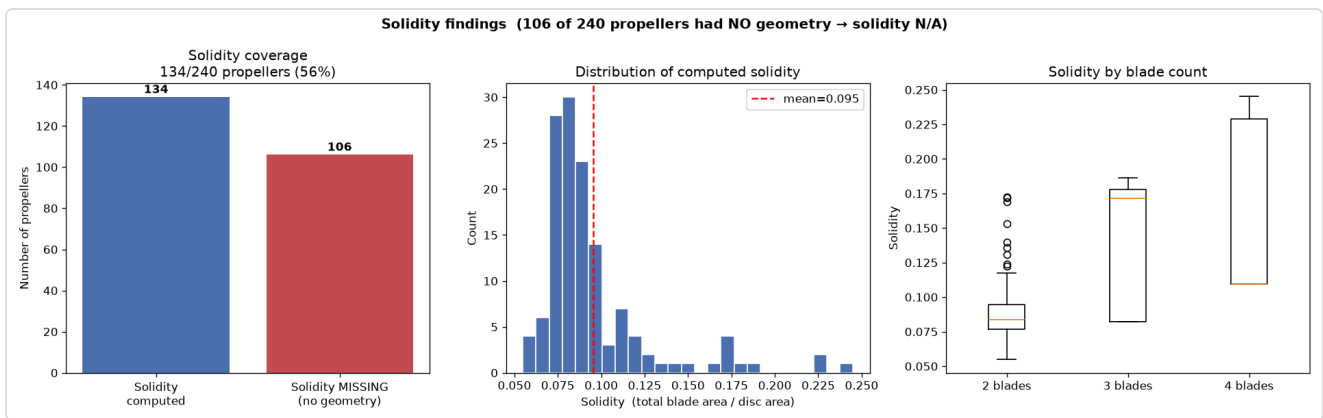


Figure — 01_solidity_findings.png

- Solidity was obtained for **134 of 240 propellers (56 %)**. The remaining **106 have no geometry rows**, so their solidity is N/A .
- Computed solidity ranges **0.055 – 0.245**, mean **0.095** — the realistic band for slender UAV propellers.
- Solidity **risks with blade count**, exactly as physics predicts (more blades ⇒ more blade area on the same disc):

Blades	Mean solidity
2	0.089
3	0.137
4	0.163

4.2 Collation

Solidity (and the blade/disc areas) are merged back onto the experiment table by `propeller_name` → `data/processed/experiment_with_solidity.csv` (27,495 × 16). **13,284 rows (48.3 %)** inherit a missing solidity — the hook for §5.

5. Week 2 — Exploratory analysis & visualisation (`src/03_edu_visualizations.py`)

- `02_target_distributions.png`

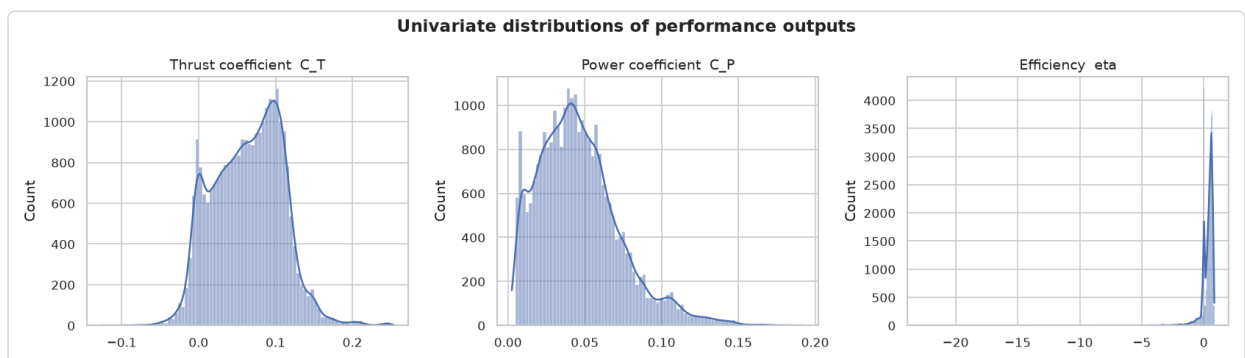


Figure — 02_target_distributions.png

— C_T and C_P are right-skewed and strictly positive in the working state; η is left-skewed toward its ~0.7–0.8 ceiling with a long negative tail from the windmill state.

- 03_performance_curves.png

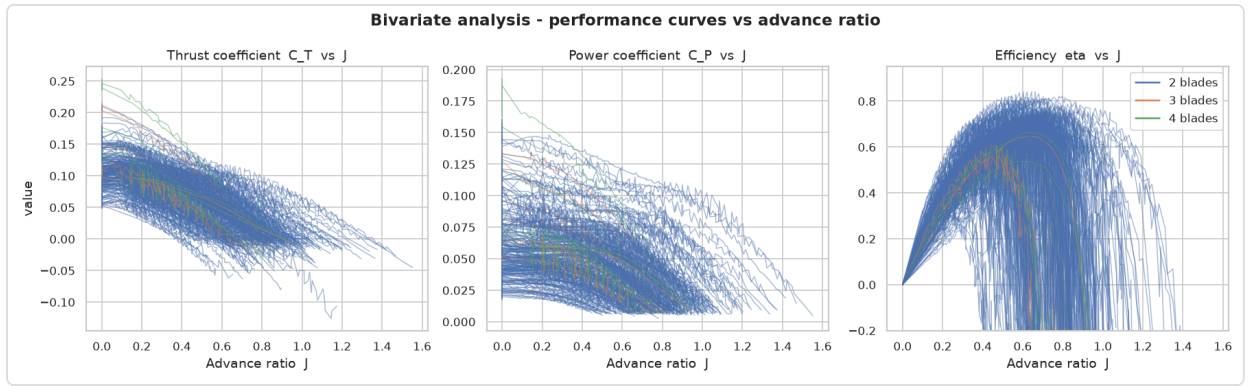


Figure — 03_performance_curves.png

— the classic propeller signatures: C_T falls almost linearly with advance ratio J (crossing zero near $J \approx p/D$), C_P falls more gently, and efficiency rises, peaks near $J \approx 0.6-0.8$, then collapses.

- 04_bivariate_drivers.png

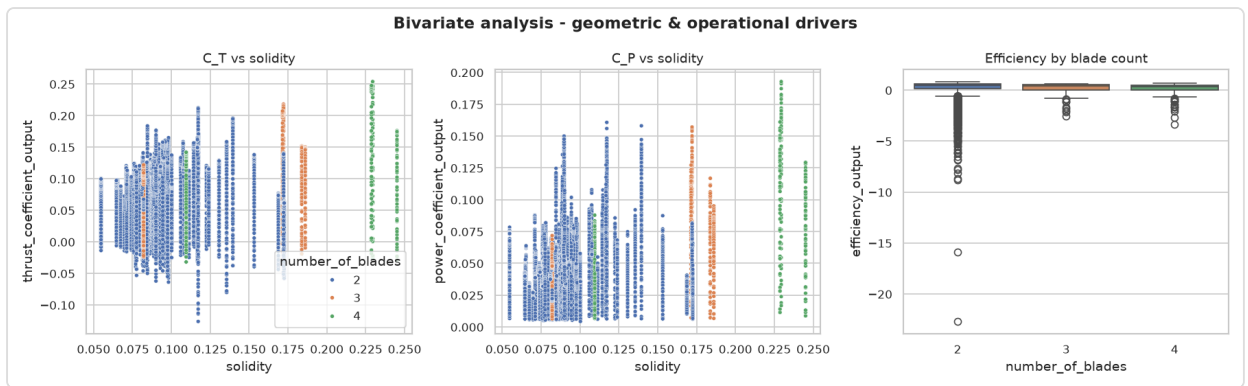


Figure — 04_bivariate_drivers.png

— C_T / C_P vs solidity and efficiency vs blade count.

- 05_correlation_heatmap.png

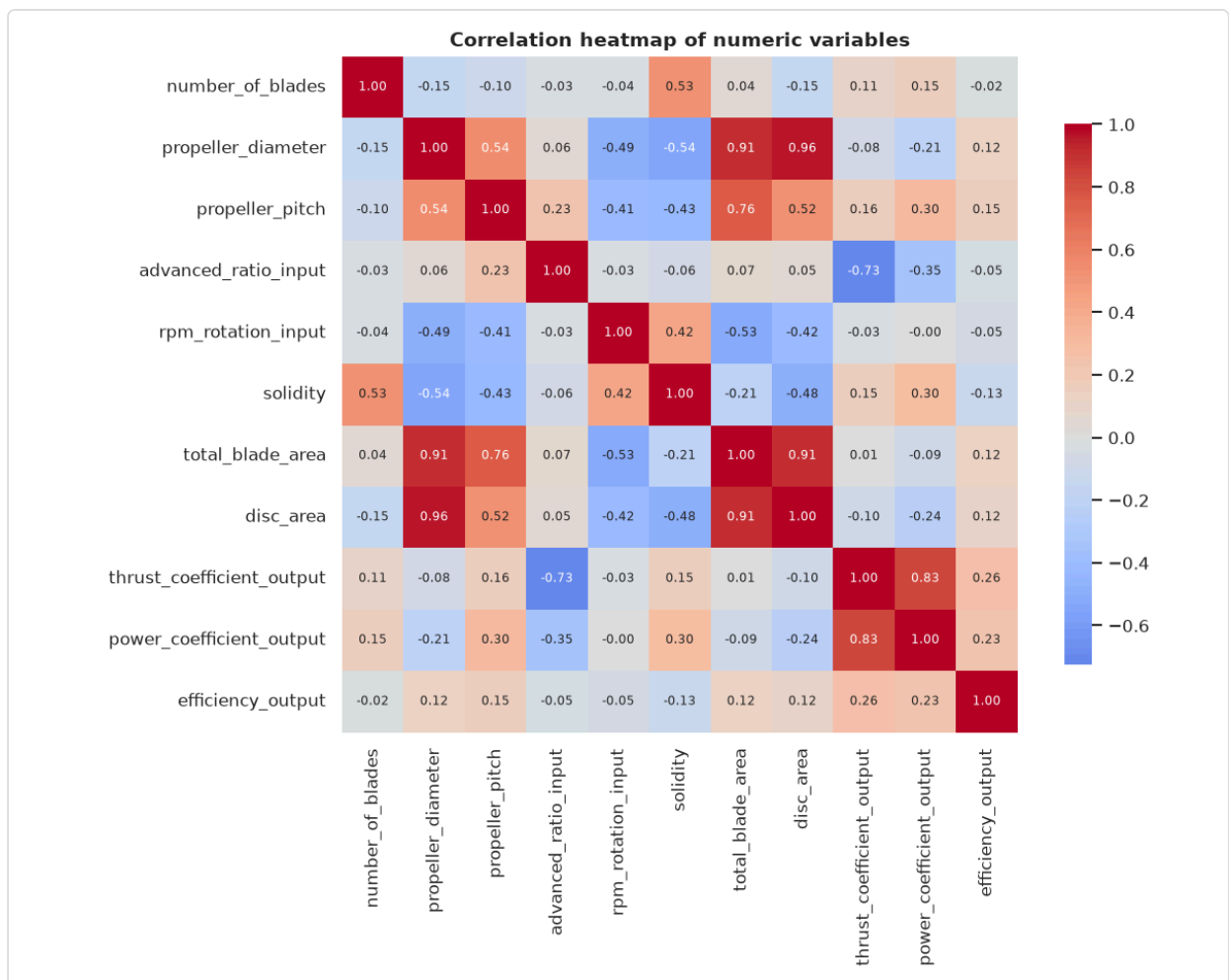


Figure — 05_correlation_heatmap.png

— headline correlations:

- $C_T - C_P$ **+0.83** (thrust and power move together),
- $C_T - J$ **-0.73** (advance ratio is the dominant operating driver),
- $\text{solidity} - \text{number_of_blades}$ **+0.53**, $\text{disc_area} - \text{diameter}$ **+0.96**.
- Linear correlations *with efficiency* look weak only because raw η is attenuated by the extreme windmill-state outliers; in the working state the physical relationships above are strong.

Bivariate takeaway: the biggest lever on the three outputs is the **operating point (advance ratio / RPM)**; among *geometry* variables, **pitch-to-diameter** and **blade count** matter most, while **solidity** is a secondary, blade-count-correlated driver.

6. Week 2 — Machine learning (src/04_ml_models.py)

6.1 Missing-value treatment

- **Audit:** the coefficients are complete; the only missing feature is **solidity** (48.3 % of rows), because 106 propellers lack geometry.
- **Efficiency outliers:** past zero-thrust, $\eta = J \cdot C_T / C_P$ explodes toward large negatives as $C_P \rightarrow 0$ (down to ≈ -22.7). These 601 rows are clipped to the physically meaningful band **[-1, 0.9]**; C_T and C_P are left untouched.

6.2 Design of the experiment

- **Technique:** Gradient Boosting (GradientBoostingRegressor , 400 trees, depth 3, lr 0.05) — the ensemble method named in the brief.

- **Train / evaluate split as instructed:** train **only on 2-blade propellers** (226 props, 26,615 rows) and **evaluate on the other propellers** — the held-out **3- and 4-blade** propellers (14 props, 880 rows). This is a genuine *extrapolation* test.
- **Features:** diameter, pitch, advance ratio, RPM (+ solidity, depending on variant). `number_of_blades` is deliberately **excluded** — it has no variance in the 2-blade training set and cannot be learned.
- **Three variants (the requested comparison):**
 - **A — Without imputation:** complete-case; drop training rows whose solidity is missing.
 - **B — With imputation:** median-impute solidity, keep all rows.
 - **C — Without solidity:** drop the solidity feature entirely.

6.3 Results — R^2 on the held-out 3- & 4-blade propellers

See `outputs/figures/06_model_comparison_r2.png`

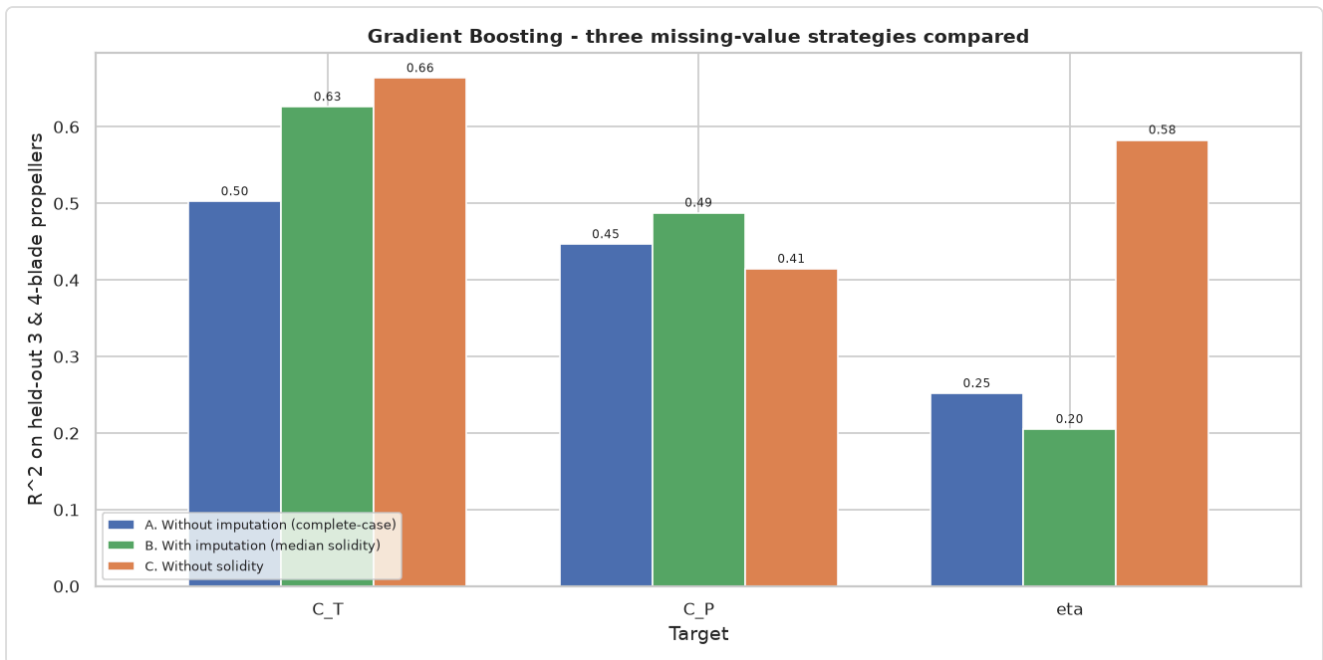


Figure — 06_model_comparison_r2.png

, `07_pred_vs_actual.png`

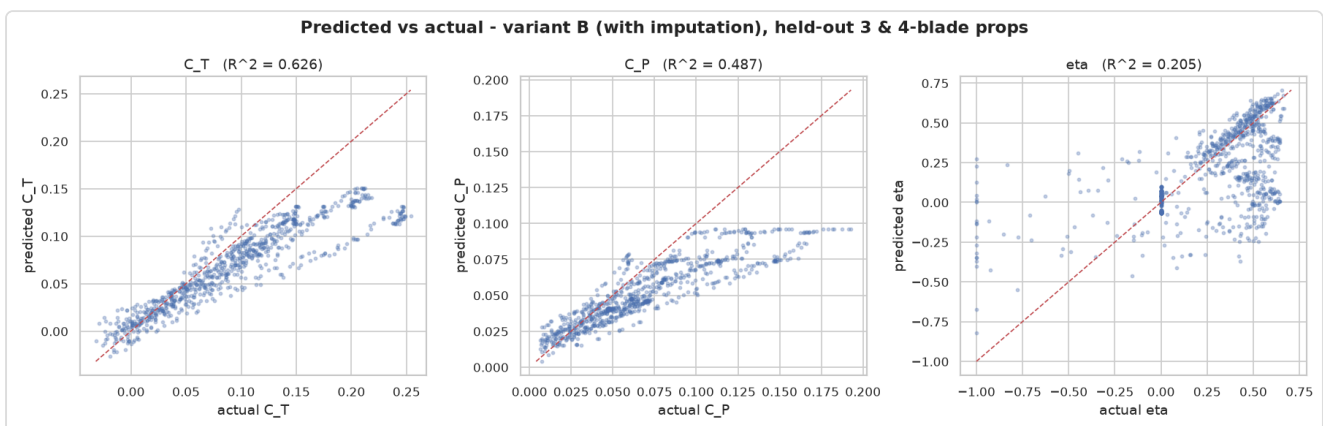


Figure — 07_pred_vs_actual.png

, `08_feature_importance.png`

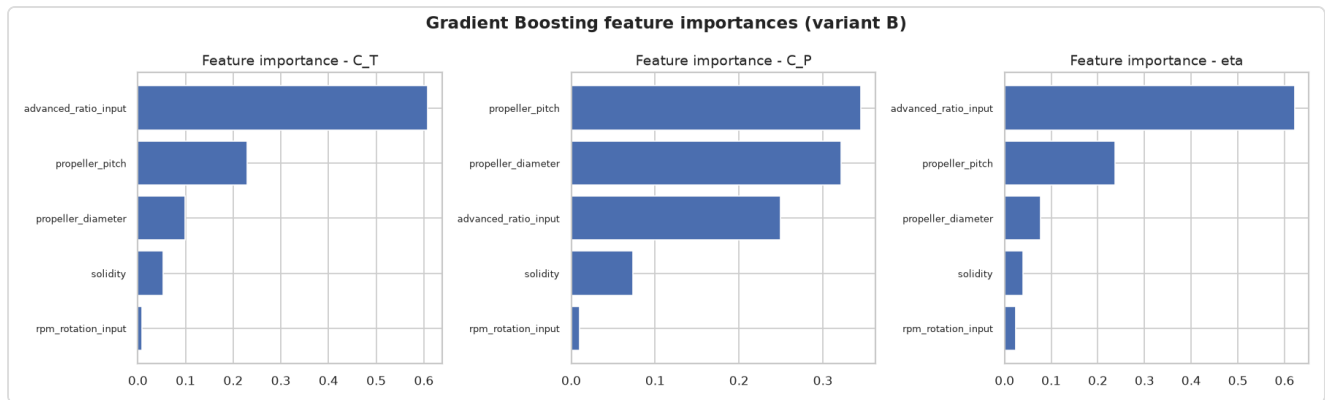


Figure — 08_feature_importance.png

, and `outputs/tables/model_comparison.csv` .

Target	A · no imputation	B · imputation	C · no solidity
C_T	0.50	0.63	0.66
C_P	0.45	0.49	0.41
η	0.25	0.20	0.58

Training-data cost: variant **A** discards half the training rows (13,331 of 26,615) and, in deployment, *cannot score any propeller that lacks geometry*. Variants **B** and **C** keep all 26,615 rows; variant **C** needs no geometry at all, so it can score every propeller ever made.

6.4 Interpretation

- **Imputation beats deletion.** Across every target, keeping the data (B) and imputing the missing solidity matches or beats throwing rows away (A) — and it restores the ability to make predictions for geometry-less propellers.
- **Solidity can hurt extrapolation.** For `C_T` and especially `$\eta$` , *dropping* solidity (C) generalises best. The reason is a distribution shift: the 2-blade training solidities (~0.09) **do not cover** the 3-/4-blade test solidities (0.14–0.16), so a median-imputed or in-range solidity feature misleads the model when it extrapolates. The most robust operational model here is the **solidity-free Gradient Boosting model (C)**.
- **Feature importance** confirms the EDA: **advance ratio dominates**, with pitch and diameter next.
- Honest limitation: extrapolating from 2-blade to 3-/4-blade propellers is intrinsically hard, so `$\eta$` is only moderately predictable; `C_T` / `C_P` reach $R^2 \approx 0.5$ –0.66.

7. Week 2 — Tableau dashboard (data storytelling)

An analyst-ready extract (`data/processed/tableau_extract.csv` , 22 fields incl. `pitch_to_diameter` , `working_state` , `peak_efficiency` , `j_at_peak_eff` , `blade_class`) plus a full build specification (`report/tableau_dashboard_spec.md`). The finished, coordinated dashboard is rendered at `outputs/figures/10_dashboard.png`

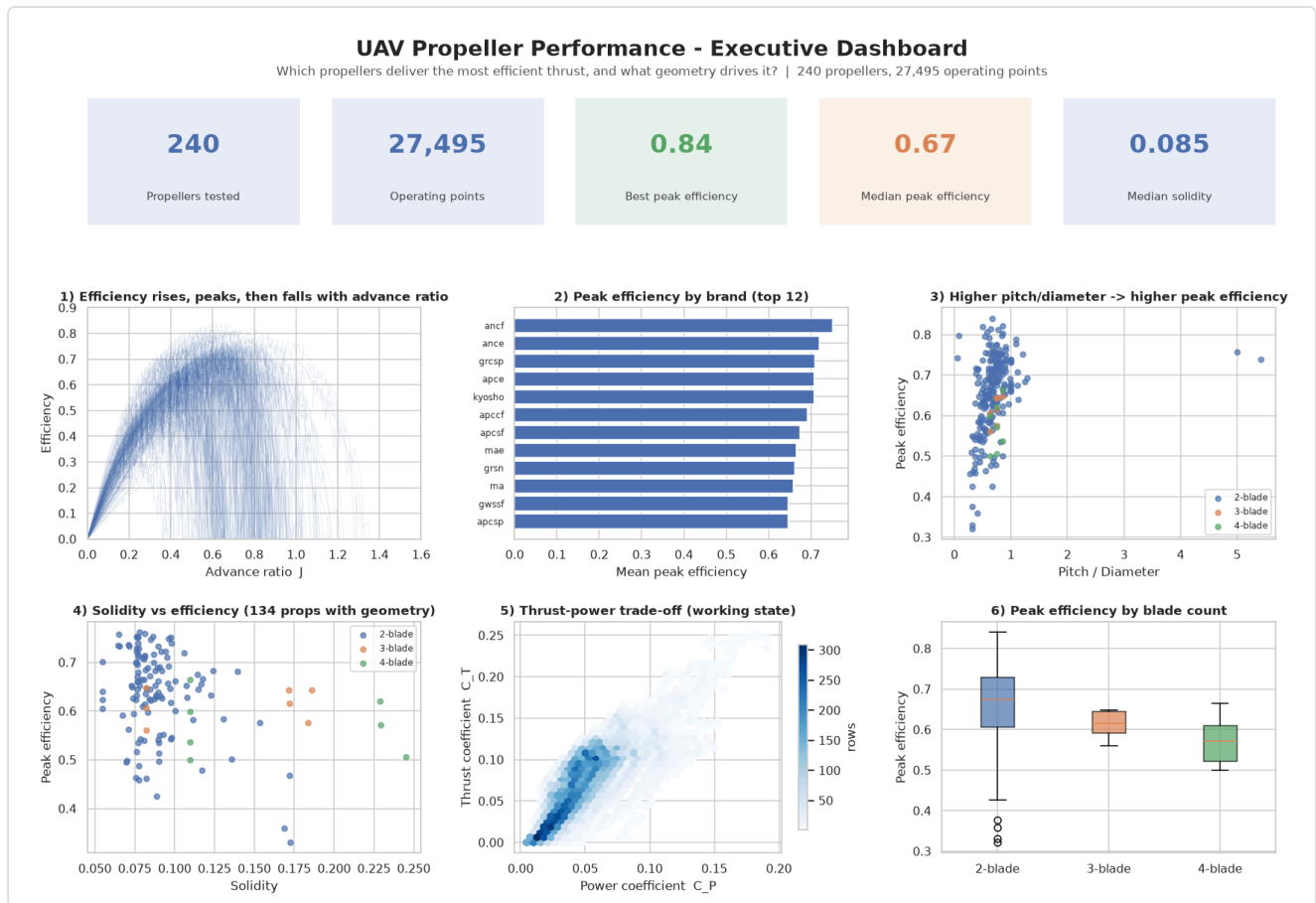


Figure — 10_dashboard.png

:

- a **KPI banner** (propellers, operating points, best/median peak efficiency, median solidity);
- the **efficiency envelope** vs advance ratio;
- **peak efficiency by brand**;
- **pitch/diameter** → **efficiency** and **solidity** → **efficiency** drivers;
- the **thrust–power trade-off** density; and
- **peak efficiency by blade count**.

Story: what we measured → what makes a propeller efficient (run it near its best advance ratio; favour higher pitch/diameter) → the 2-blade sweet spot for these small-UAV propellers.

8. Conclusions & recommendations

1. A **gradient-boosted ensemble** predicts propeller thrust and power well ($R^2 \approx 0.5\text{--}0.66$) even when extrapolating to unseen blade counts, and reduces the need for exhaustive wind-tunnel sweeps.
2. **Handle missing geometry by imputation or by a solidity-free model — never by deletion.** Deletion halves the data and blinds the model to any propeller without a measured blade shape.
3. **Operating point is king.** Advance ratio (hence flight speed vs RPM) is the strongest predictor; select props and set RPM to sit near the efficiency peak ($J \approx 0.6\text{--}0.8$).
4. **Geometry levers:** higher pitch/diameter lifts peak efficiency; blade count and solidity trade thrust density for efficiency (2-blade props are the efficiency sweet spot in this database).
5. **Next steps:** capture the missing geometry for the 106 uncovered propellers; add Reynolds-number / airfoil features; and model η only in the working state ($C_T > 0$) for a cleaner target.

Appendix — how to reproduce

```
pip install -r requirements.txt
cd src
python 01_data_preparation.py      # append + rename + distributions
python 02_solidity_analysis.py     # trapz areas, disc area, solidity, collation
python 03_eda_visualizations.py    # EDA + correlation heatmap
python 04_ml_models.py            # Gradient Boosting, 3 variants
python 05_run_sql.py              # Week-1 SQL answers
python 06_tableau_dashboard.py    # Tableau extract + dashboard render
# or simply: python run_all.py
```

All figures land in `outputs/figures/` , all result tables in `outputs/tables/` .